Technische Universität Berlin
Faculty IV: Electrical Engineering and Computer Science
Freie Universität Berlin
Department of Mathematics and Computer Science
Algorithmic Bioinformatics



Automatic Optimization of Level Parameters for RSHash

(RSHash is a minimizer based fast, compressed hash table for storing DNA kmers. It was recently introduced by the Reinert Group.)

Bachelor's Thesis

by Bjarne David Jehn

to obtain the degree of "Bachelor of Science" (B. Sc.)
in the Computer Science degree studies (Informatik)

First Reviewer:	Prof. Dr. Knut Reinert
Second Reviewer:	Prof. Dr. Stephan Kreutzer
Supervisor:	Jonas Schulte-Mattler

Declaration of Authorship

I hereby declare that the thesis submitted is my own, unaided work, completed without any external help. Only the sources and resources listed were used. All passages taken from the sources and aids used, either unchanged or paraphrased, have been marked as such.

Where generative AI tools were used, I have indicated the product name, manufacturer, the software version used, as well as the respective purpose (e.g. checking and improving language in the texts, systematic research). I am fully responsible for the selection, adoption, and all results of the AI-generated output I use.

I have taken note of the Principles for Ensuring Good Research Practice at TU Berlin dated 15 February 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

I further declare that I have not submitted the thesis in the same or similar form to any other examination authority.

Berlin, _____ _____

Date Signature

Abstract

Contents

1 Motivation 5

2 Minimizer Distribution 6

3 MINLP Modeling 7

4 Subsampling 13

1 Motivation

2 Minimizer Distribution

The MINLP Modeling only works, if the values $\mathbf{p_K}(m,t)$ are known for every $(m,t) \in \mathcal{M} \times \mathcal{T}$ for the input sequence.

Determining the Distribution

The code to find the minimizer distribution for a given $sequence, k, \mathbf{m}$ and $\mathbf{t}$ follows the following steps:
stats:

1. $count_minimizers$:
   a) set $kmers \leftarrow |sequence| - k + 1$
   b) parse every k-mer in the $sequence$ and add all minimizers to a vector $minimizers$
   c) sort $minimizers$
   d) iterate over the sorted vector and
      - count the occurences of each minimizer
      - add all unfrequent minimizers to a new vector $unfrq_minimizers$
      - store their occurences in the vector $counts$ where $counts[i]$ relates to $unfreq_minimizers[i]$
      - store the sum of all occurences in no_skmers
2. $minimizers \leftarrow unfreq_minimizers$
3. $no_minimizers \leftarrow |minimizers|$
4. $r \leftarrow$ elias-fano-compressed bitvector of length $2^{\mathbf{2m}}$ that contains 1s at those positions i, for which the minimizer of value i is present in $minimizers$
5. $s \leftarrow$ corresponding vector S
6. weiter ab Zeile 161

Distribution on Higher Levels

3 MINLP Modeling

One Level

As a first approach we model the runtime and space consumption for only one level.

Notation

The minimizer of a k-mer q is reffered to as $m(q)$ and its frequency as $f(m(q))$.
We denote the time needed to access bit in R with T_{access_R}, the time for rank and select queries with T_{rank_R}, T_{select_R} and analogously for the vectors S and O.
The time for checking whether a k-mer is present at a given offset in the Text is denoted as $T_{compare_T}$.
Let $\mathcal{M}$ be the Set of all minimizer length and $\mathcal{T}$ the set of all thresholds to be considered when searching for optimal parameters.
We denote the optimal minimizer length $\mathbf{m}$ and threshold $\mathbf{t}$.
Let $\mathbf{p_K}$ be the percentage of k-mers q with unfrequent minimizers of length $\mathbf{m}$ ($f(m(q)) \leq \mathbf{t}$). $P_K(m,t)$ on the other hand is the empirically measured percentage on a fixed sequence and a given pair $(m,t) \in \mathcal{M} \times \mathcal{T}$.

Runtime

Vector Access

Bit Vector Rank Query

Bit Vector Select Query

Lookup

A k-mer q is looked up by first computing its minimizer $m(q)$. If the bit $R[m(q)]$ is not set, q is not in the data structure.
For the $\mathbf{p_K}$ of the k-mers for which the bit is set, the minimizer's rank $r = rank_R(m(q))$. Then $select_S(r)$ and $select_S(r+1)-1$ tell the index in Offsets O where the minimizer's first and last offset in the text T are stored.
Finally the presence of q in T is checked at every offset $o \in O[select_S(r) : select_S(r+1)-1]$. For the lookup case, this is terminated as soon as q is found for the first time, which is on average at offset $O[\frac{select_S(r+1)-1-select_S(r)}{2}]$.

For $1 - \mathbf{p_K}$ $m(q)$ does not appear at all or has a $f(m(q)) > t$. In this case the lookup is completed through an access of the hash table.

The lookup time $T_{lookup_{RSHash}}$ for one level can thus be modeled as:

$$T_{lookup_{RSHash}} = T_{access_R} + \mathbf{p_K} \cdot (T_{rank_R} + 2T_{select_S} + \frac{\mathcal{F}^{\mathbf{t}}_{\mathbf{m}}}{2}) + (1 - \mathbf{p_K}) \cdot T_{access_H}$$

In order to determine m and t we introduce binary selector variables μ_m, τ_t for each possible $m \in \mathcal{M}$ and $t \in \mathcal{T}$.
μ_m should be 1 exactly when the optimal minimizer length is m.
τ_t should be 1 exactly when the optimal threshold is t.
The selectors for m and t must be binary and mutually exclusive:

$$\begin{aligned} \mu_m &\in 0, 1 \mid \forall m \in \mathcal{M} \\ \tau_t &\in 0, 1 \mid \forall t \in \mathcal{T} \\ \sum_{m \in \mathcal{M}} \mu_m &= 1 \\ \sum_{t \in \mathcal{T}} \tau_t &= 1 \end{aligned}$$

With $P_K(m, t)$ being the empirically determined percentage of k-mers q that have a minimizer with $f(m(q)) < t$ (for a fixed $(m, t) \in \mathcal{M} \times \mathcal{T}$) we can then model $\mathbf{p_K}$ with:

$$\mathbf{p_K} = \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} P_K(m, t) \cdot \tau_t \right)$$

$\mathbf{m}$ and $\mathbf{t}$ can then be determined by:

$$\begin{aligned} \mathbf{m} &= \sum_{m \in \mathcal{M}} \mu_m \cdot m \\ \mathbf{t} &= \sum_{t \in \mathcal{T}} \tau_t \cdot t \end{aligned}$$

The average Frequency $\mathcal{F}^{\mathbf{t}}_{\mathbf{m}}$ can be ...

$$\mathcal{F}^{\mathbf{t}}_{\mathbf{m}} = \frac{P_K(\mathbf{m}, 1) + \sum_{f=2}^{\mathbf{t}} P_K(\mathbf{m}, f) - P_K(\mathbf{m}, f-1) * f}{\sum_{f=1}^{\mathbf{t}} f}$$

The above results in the following MINLP (Mixed Integer Non Linear Program):

MINIMIZE

$$T_{lookup_{RSHash}} = T_{access_R} + \mathbf{p_K} \cdot (T_{rank_R} + 2T_{select_S} + \frac{\mathcal{F}_{\mathbf{m}}^{\mathbf{t}}}{2}) + (1 - \mathbf{p_K}) \cdot T_{access_H}$$

SUBJECT TO

$$\begin{aligned}
\mathbf{p_K} &= \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} P_K(m,t) \cdot \tau_t \right) \\
\sum_{m \in M} \mu_m &= 1 \\
\sum_{t \in T} \tau_t &= 1 \\
\mu_m &\in 0,1 \mid \forall m \in M \\
\tau_t &\in 0,1 \mid \forall t \in T \\
\mathbf{m} &= \sum_{m \in \mathcal{M}} \mu_m \cdot m \\
\mathbf{t} &= \sum_{t \in \mathcal{T}} \tau_t \cdot t
\end{aligned}$$

Solving this MINLP will result in a Solution where the μ_m, τ_t corresponding to the smallest $m \in \mathcal{M}$, $t \in \mathcal{T}$ will be set to one.
This is as expected, since this leads to the smallest possible $\mathbf{p_K}$. This minimizes the k-mers found in the vector S and thus maximizes the lookups in the (faster) hash table. This however results in maximal space consumption, as each entry in the hash table is significantly larger than an entry in S as described in Motivation.

In order to take that into account, we have to add constraints modeling the space consumption.

Space Consumption

The space consumption of the Data Structure used by RSHash can be split into that of the Vectors R, O and S as well as the hash table H.

Bit Vectors R and S

R and S can likely be disregarded when modeling the space consumption. R is a Bit Vector of length $4^{\mathbf{m}}$ that contains n

$$n\lceil log(4^{\mathbf{m}}/n) \rceil + 3n$$

Offsetvector O

O contains one entry of size $log|T|$ for the offset of every minimizer with frequency $\leq t$ in T.
Every minimizer covers an average of $(k - \mathbf{m} + 2)/2$ consecutive k-mers. We can thus model the bits per k-mer in O as:

$$bpk_O = \frac{\mathbf{p_K}}{(k - \mathbf{m} + 2)/2} \cdot log|T|$$

Hash table H

For $1 - \mathbf{p_K}$ of the k-mers q in T H contains an entry $(key : q, value : offset(q))$
Each entry has size $size(key) + size(value) = 64 + log|T|$. We introduce the load factor α to the model, in order to account for empty buckets in the hash table.
The bits per k-mer required for the hash table can thus be modeled by

$$bpk_H = (1 - \mathbf{p_K}) \cdot \frac{(64 + log|T|)}{\alpha}$$

Several Levels

Runtime

RSHash uses several levels, to make use of the skewniss of minimizer distribution.
Each level l has a pair of parameters $\mathbf{m}^l, \mathbf{t}^l$ which result in a probability $\mathbf{p}^l_\mathbf{K}$ of a k-mer belonging to level l.
For now, we still assume, that the time for a lookup in the vectors R, S as well as the hash table H are (practically) independent of the parameters $\mathbf{m}^l, \mathbf{t}^l$.
When using L levels, the probability of a k-mer belonging to a higher level is dependent on the probability of k-mers belonging to the levels below.
On each level $1 \leq l \leq L$, for $\mathbf{p}^l_\mathbf{K}$ of the k-mers, the lookup will be completed by finding a value in the offset vector on that level whilst the remaining $(1 - \mathbf{p}^l_\mathbf{K})$ queries result in a lookup on the next level.
We can model the lookup time for all levels $1 \leq l \leq L$ levels recursively as:

$$T_{lookup_{RSHash}} = \begin{cases} T_{access_{R^l}} + \mathbf{p}^l_\mathbf{K} \cdot (T_{rank_{R^l}} + 2T_{select_{S^l}} + \frac{\mathcal{F}^\mathbf{t}_\mathbf{m}}{2}) + (1 - \mathbf{p}^l_\mathbf{K}) \cdot T^{l+1}_{lookup_{RSHash}} & | \, 1 \leq l \leq L \\ T_{access_H} & | \, l = L + 1 \end{cases}$$

As an example for $L = 2$ levels, this results in:

$$\begin{aligned} T_{lookup_{RSHash}} = T_{access_{R^1}} \quad & + \mathbf{p}^1_\mathbf{K} \cdot (T_{rank_{R^1}} + 2T_{select_{S^1}} + \tfrac{\mathcal{F}^\mathbf{t}_\mathbf{m}}{2}) \\ & + (1 - \mathbf{p}^1_\mathbf{K}) \cdot \Big(T_{access_{R^2}} + \quad \mathbf{p}^2_\mathbf{K} \cdot (T_{rank_{R^2}} + 2T_{select_{S^2}} + \tfrac{\mathcal{F}^\mathbf{t}_\mathbf{m}}{2}) \\ & \qquad + (1 - \mathbf{p}^2_\mathbf{K}) \cdot T_{access_H} \Big) \end{aligned}$$

Because $\mathbf{m}^l$ and $\mathbf{t}^l$ can be different on every level l, a different set of binary selector variables μ^l_m and τ^l is needed.

The constraints for each level are the same as for one level:

$$\begin{aligned} \mu_m^l &\in 0,1 && \mid \forall (l,m) \in (\{1,...,L\} \times \mathcal{M}) \\ \tau_t^l &\in 0,1 && \mid \forall (l,t) \in (\{1,...,L\} \times \mathbf{t}) \\ \sum_{m \in \mathcal{M}} \mu_m^l &= 1 && \mid \forall l \\ \sum_{t \in \mathcal{T}} \tau_t^l &= 1 && \mid \forall l \end{aligned}$$

For every level l $\mathbf{p_K^l}, \mathbf{m}^l$ and $\mathbf{t}^l$ can also be modeled analogously to having only one level:

$$\begin{aligned} \mathbf{p_K^l} &= \sum_{m \in \mathcal{M}} \left(\mu_m^l \sum_{t \in \mathcal{T}} P_K(m,t) \cdot \tau_t^l \right) && \mid \forall l \in \{1,...,L\} \\ \mathbf{m}^l &= \sum_{m \in \mathcal{M}} \mu_m^l \cdot m && \mid \forall l \in \{1,...,L\} \\ \mathbf{t}^l &= \sum_{t \in \mathcal{T}} \tau_t^l \cdot t && \mid \forall l \in \{1,...,L\} \end{aligned}$$

Space Consumption

Offsetvector O

For several levels, the offset vector for level l only contains an entry for a k-mer q, if the k-mers minimizer $m(q)$ has a frequency $f(m(q)) > t^i$ for all levels $1 \leq i < l$ and $f(m(q)) \leq t^l$. The bits per k-mer for the Offset vector on level l can thus be modeled as:

$$bpk_{O^l} = \prod_{i=1}^{l-1} (1 - \mathbf{p_K^i}) \cdot \mathbf{p_K^l} \cdot \frac{2}{k - \mathbf{m}^l + 2} \cdot log|T|$$

Hash table H

The hash table only contains an entry k-mers q, for which $f(m(q)) > t^l$ for all levels $1 \leq l \leq L$.
The bits per k-mer for the hash table can thus be modeled as

$$bpk_H = \prod_{l=1}^{L} (1 - \mathbf{p_K^l}) \cdot (64 + log|T|)$$

Limitations

This approach of solving the constructed model has the advantage (However for large genomes, finding the empirical values $P_K(m,t)$ becomes increasingly time consuming and needs a lot of RAM

For these reasons, an approach yielding a solution that is a certain approximation of the optimum but takes less time to find it could be of interest.
An easy adaption to speed up the search for empirical values would be to only sample part of the genome. By sampling sequences of length By sampling only part of the genome, we can speed up the finding of empirical values by a factor of

4 Subsampling

Motivation

The time for processing a genome in order to find the minimizer distribution $\mathbf{p_K}(m, t)$ for possible minimizer lengths $m \in \mathcal{M}$ and thresholds $t \in \mathcal{T}$ is directly proportional to the size of the genome.
For large genomes, this processing can become the dominating factor when searching for optimal parameters using the MINLP Modeling approach. In order to speed up the search for the empirical values, we introduce a subsampling method.

Subsampling Method

Instead of calculating the distribution over the full genome, we sample n sequences of length $l \geq k$ at random positions in the genome.

Time Complexity of Finding the Minimizer Distributions

Time Complexity using Subsampling

By applying the proposed subsampling strategy, the original genome T of length $|T|$ is reduced to the subsampled text T^S of length $|T^S| = n \cdot l$. This means a reduction in size by a factor of $\frac{|T|}{|T^S|}$.Depending on the choice of n and l this results in a speedup of the processing time by a factor of

Error Analysis

Choosing l

The length l has to be at least k.

Choosing n

Choosing a constant n would result in a processing time that would no longer be dependent of the genomes length $|T|$, as the sampled text would have constant size $n \cdot l$If instead n is chosen relative to $|T|$, $\frac{|T^S|}{|T|}$ will have a constant value independent of the genomes size. Even though this means, that the time needed, in order to find the minimizer distributions would still be proportional to the genomes size, this should be the preferred option as it results in a bounded error of the approximation.
For a large enough n, the probability of choosing a bad representation becomes highly unlikely.